

ЛЕКЦИЯ № 1

Вводная лекция

инструменты, железо и первый код

Автор: Леонтьев Д.А.

Вы уже умеете писать код

Но ответьте на простой вопрос:

```
x = 5
```

- Где живёт эта пятёрка?
- Сколько места она занимает?
- Что с ней происходит, когда функция заканчивается?

Три реальных ситуации

Зачем это знать — три ситуации из жизни

1. Собеседование

Junior

«Чем отличается стек от кучи?»

Стандартный вопрос на Junior C# позицию.

Половина кандидатов не отвечает.

2. Баг в продакшене

Программа работает нормально, но раз в несколько минут зависает на 2 секунды.

Логи чистые. Сервер в норме. Всё выглядит нормально.

Кто знает железо

диагноз за 30 секунд

Кто не знает

ищет неделю

3. «Почему так медленно?»

Два варианта кода. Делают одно и то же.

x8

Один быстрее другого
в 8 раз.

Разница — в одной строчке.

Не зная железо — невозможно угадать, где проблема.

Прошлый год

Проблемы и опыт

Рекомендуемая литература

Для новичков и продвинутых

Книги для новичков



Тайный язык информатики



Чарльз Петцольд

Книга для вдохновения

Рэндал Э. Брайант
Дэвид Р. О'Халларон

Компьютерные системы

Архитектура и программирование

Третье издание



МК
ИЗДАТЕЛЬСТВО

Книги для про

КЛАССИКА COMPUTER SCIENCE

АРХИТЕКТУРА КОМПЬЮТЕРА 6-Е ИЗДАНИЕ



Э. ТАНЕНБАУМ Т. ОСТИН

PH
PTR

ПИТЕР®

Sixth Edition

John L. Hennessy | David A. Patterson

COMPUTER ARCHITECTURE

A Quantitative Approach



МК
MORGAN KAUFMANN

Цифровая схемотехника и архитектура компьютера: RISC-V



Дэвид М. Харрис
Сара Л. Харрис



МИЭМ

ДМК
ИЗДАТЕЛЬСТВО

Что вы сделаете в этом курсе

Практические задачи курса

Железо и мониторинг



Разберёте и соберёте настоящий ПК

Подержите в руках CPU, RAM, материнскую плату



Напишете системный монитор

Программа, которая читает температуру CPU и загрузку памяти



Заставите все ядра работать на 100%

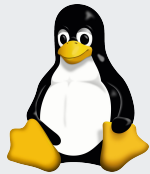
И увидите это в Task Manager в реальном времени

Код и инструменты



Откроете .exe файл в HEX-редакторе

Увидите свой код как набор байт



Запустите свой код на Linux

Без GUI, только терминал



Найдёте узкое место с помощью профайлера

И ускорите программу в несколько раз

ИТОГ

Шесть практических задач курса

#	Задача
1	Разобрать и собрать настоящий ПК
2	Написать системный монитор (температура CPU, RAM)
3	Открыть .exe в HEX-редакторе и увидеть байты
4	Загрузить все ядра на 100% и наблюдать в Task Manager
5	Запустить код на Linux без GUI, только терминал
6	Найти узкое место профайлером и ускорить программу

Структура курса

7 Лекций · 15 Практик · 7 Домашних работ

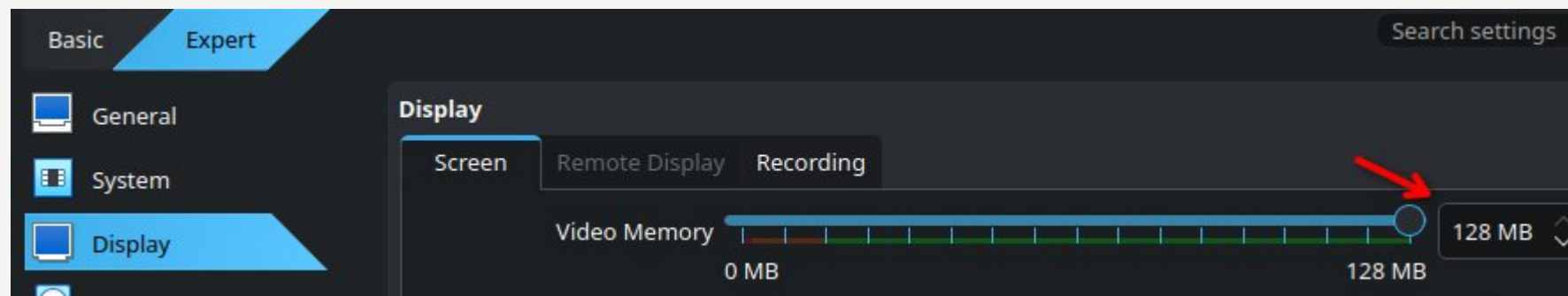
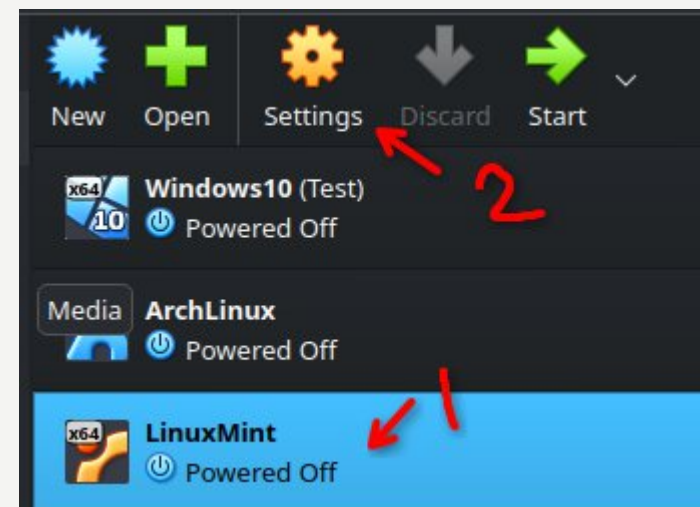
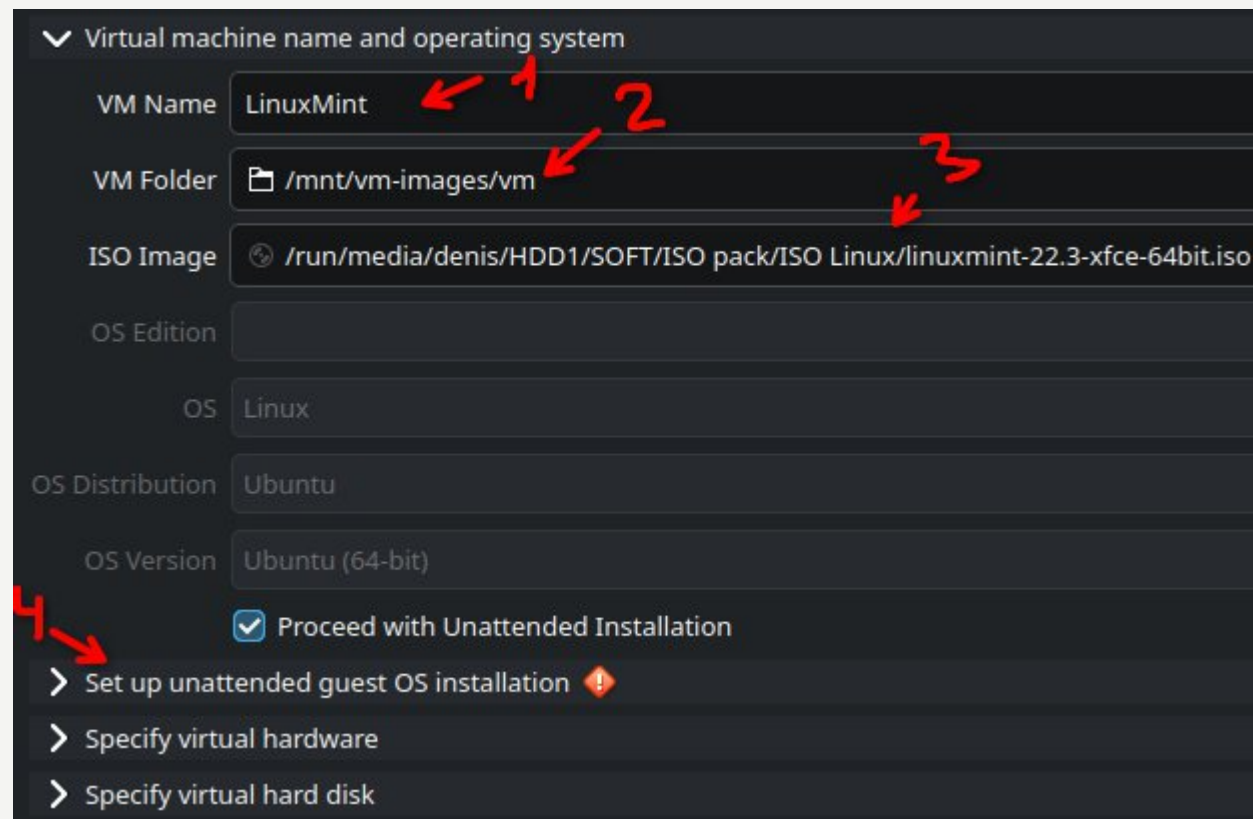
Лекции

#	Лекция
1	Вводная лекция: инструменты, железо и первый код
2	Как компьютер выполняет программу: от кода до электричества
3	Процессор: мозг компьютера и почему он не всемогущ
4	Память: иерархия, управление и оптимизация
5	Представление данных в компьютере: от битов до мультимедиа
6	Компиляция и выполнение программ: от исходного кода до машинных инструкций
7	Операционная система: Управление ресурсами и абстракция железа

Практические работы

#	Практика
1-6	Основы C#
7-12	Файлы, процессы, железо
13	Мини-проект: Системный монитор
14	Мини-проект: Анализатор файлов
15	Разборка и сборка ПК

Домашние задания — Linux Mint в VirtualBox



Who are you?

Your name: ✓

Your computer's name: ✓
The name it uses when it talks to other computers.

Pick a username: ✓

Choose a password: ☐ Short password

Confirm your password: ✓

☒ Log in automatically

☐ Require my password to log in

☐ Encrypt my home folder

Back

Continue



Please remove the installation medium, then press ENTER:

```
leontev@mint-pc:~$ sudo apt install git
[sudo] password for leontev:
Sorry, try again.
[sudo] password for leontev:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  git-man liberror-perl
Suggested packages:
  git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk
  git-cvs git-mediawiki git-svn
The following NEW packages will be installed:
  git git-man liberror-perl
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 4,806 kB of archives.
After this operation, 24.5 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y ←
```

```
leontev@mint-pc:~/projects/hw2-d2$ tree
.
├── notes.md
├── screenshots
├── work
│   ├── backup
│   ├── code
│   │   └── main.cs
│   └── docs
│       ├── file1.txt
│       └── file2.txt
6 directories, 4 files
```

drwxr-xr-x	2	leontev	leontev	4.0K	Mar 15 10:19	Desktop
права	количество жёстких ссылок	владелец	группа	размер	дата	имя

```
leontev@mint-pc:~/projects/hw2-d2$ cat > work/docs/readme.txt<<'EOF'
> Домашняя работа Д2
> Студент: Иванов Иван
> Тема: Терминал Linux
> EOF
```



```
leontev@mint-pc:~/Downloads$ ls
code_1.111.0-1772846623_amd64.deb  gcm-linux-x64-2.7.3.deb
leontev@mint-pc:~/Downloads$ sudo dpkg -i code_1.111.0-1772846623_amd64.deb
```

```
-rw-rw-r-- 1 leontev leontev 30 Mar 17 02:38 public.txt
```

```
-rw-rw-r-- 1 leontev leontev 30 Mar 17 02:38 public.txt
```

```
| _____
```

```
| права (9 символов = 3 группы по 3)
```

```
|
```

```
| r = read (чтение)
```

```
| w = write (запись)
```

```
| x = execute (выполнение)
```

```
| - = нет этого права
```

```
| [rw-] [rw-] [r--]
```

```
| | | _____ остальные: только чтение
```

```
| | | _____ группа: чтение + запись
```

```
| | _____ владелец: чтение + запись
```

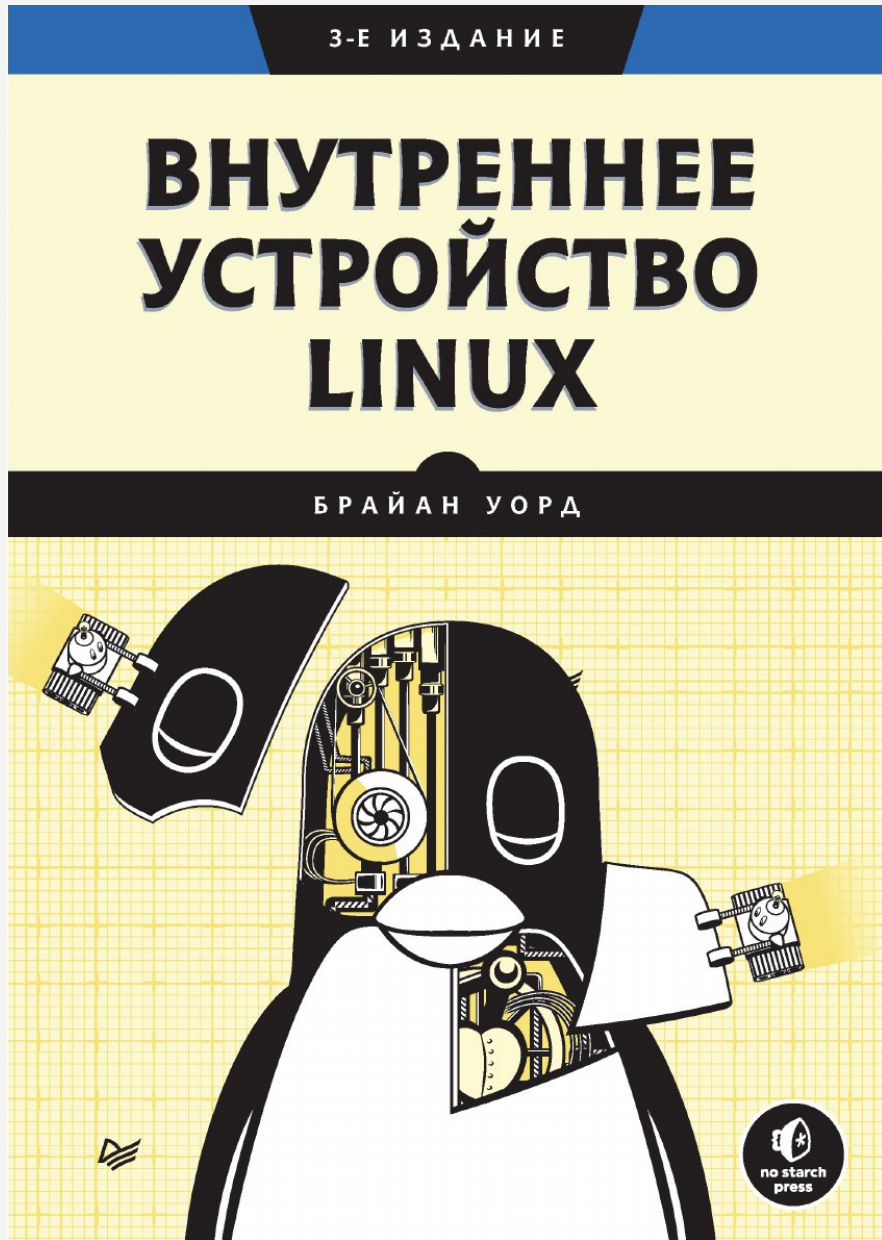
```
|
```

```
| _____ тип: - = файл, d = папка, l = символическая ссылка
```

```
leontev@mint-pc:~/projects/hw-d4$ ps aux | head
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.3	22608	13928	?	Ss	02:36	0:00	/sbin/init splash
root	2	0.0	0.0	0	0	?	S	02:36	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	02:36	0:00	[pool_workqueue_release]
root	4	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/R-rcu_gp]
root	5	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/R-sync_wq]
root	6	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/R-kvfree_rcu_reclaim]
root	7	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/R-slub_flushwq]
root	8	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/R-netns]
root	11	0.0	0.0	0	0	?	I<	02:36	0:00	[kworker/0:0H-events_highpri]

Книги для новичков по Linux



Балльно-рейтинговая система

За что получаем баллы

Лекции, посещение							Лекции, конспекты						
1	2	3	4	5	6	7	1	2	3	4	5	6	7
0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5	0,5

Практические работы + защита														
1	2	3	4	5	6	7	8	9	10	11	12	13+3	14+3	15+3
4	4	4	4	4	4	4	4	4	4	4	4	9	10	5

Домашние работы							
1	2	3	4	5	6	7	Итог
3	3	3	3	3	3	3	100,00

7 лекций + конспекты = $7 * 0,5 + 7 * 0,5 = 7 \text{ б.}$

12 практических работ + защита = $1 * 12 + 3 * 12 = 48 \text{ б.}$

3 итоговых работ + защита = $9 + 10 + 5 = 24 \text{ б.}$

7 домашних работ = $7 * 3 = 21 \text{ б.}$

Бонус = 10 б. (+сверхлимитная система)

-50%, если работы делаются не на парах. Более тщательный опрос

Связь с будущим курсом

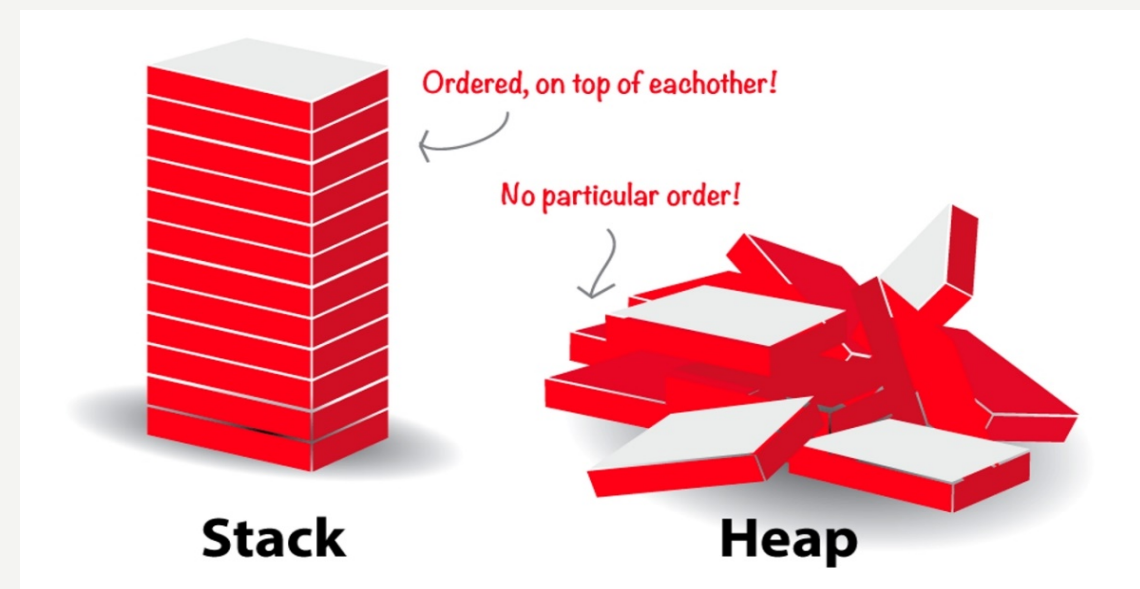
Что будет во 2 семестре

```
1 namespace OOPBasics
2 {
3     0 references
4     internal class Program
5     {
6         0 references
7         static void Main(string[] args)
8         {
9             Console.WriteLine("Hello, World!");
10        }
11    }
12
13    0 references
14    class Student
15    {
16        string? name;
17        int age;
18        string? group;
19
20        0 references
21        public void PrintInfo()
22        {
23            Console.WriteLine($"Имя: {name}, Возраст: {age}, Группа: {group}");
24        }
25    }
```

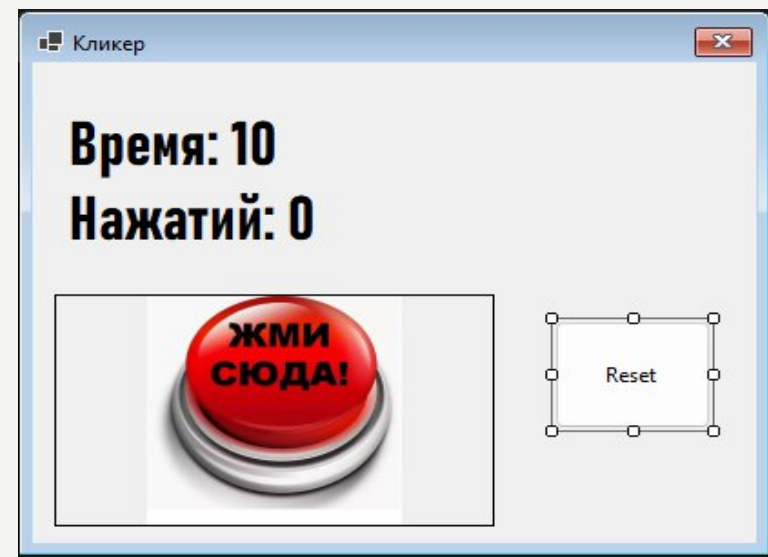
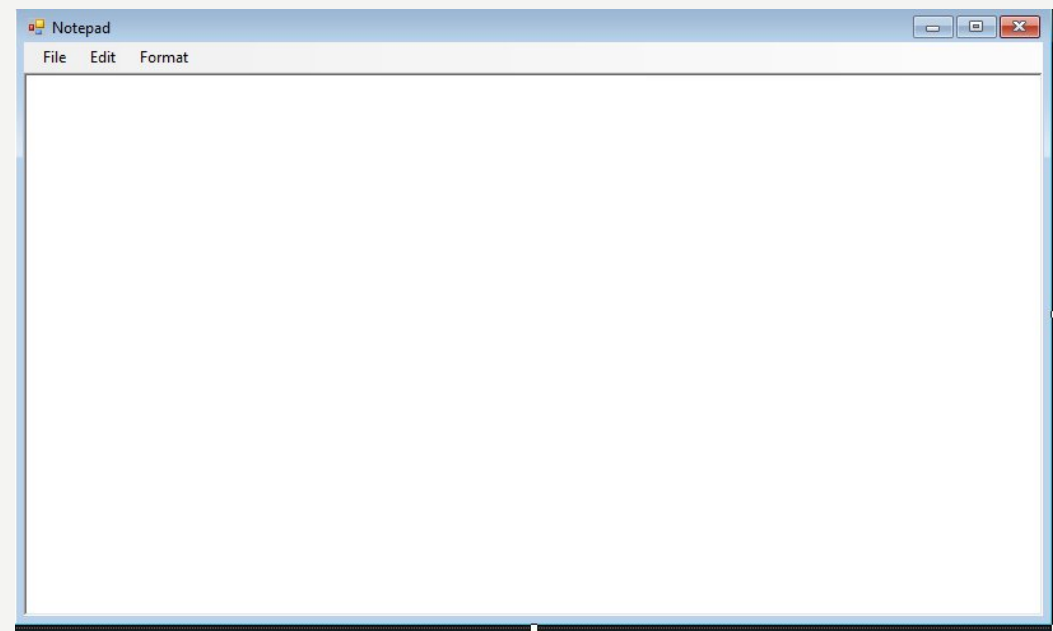
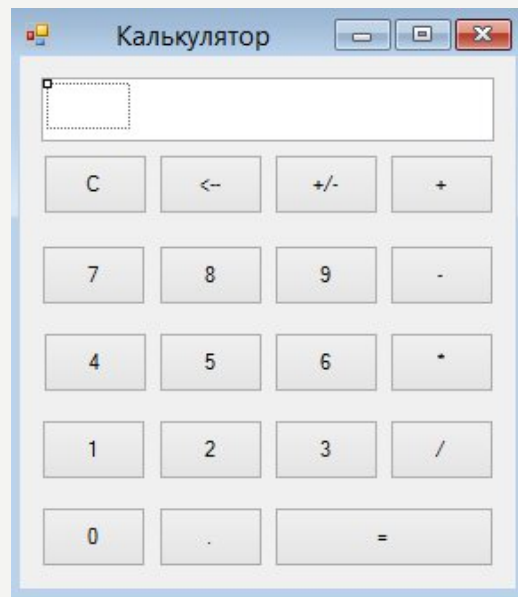
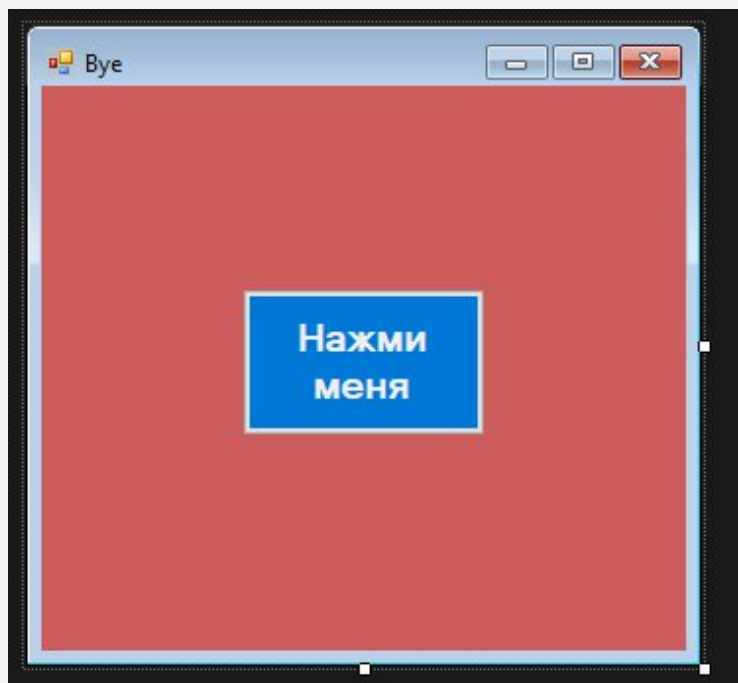
```
class Person
{
    private string name = "Undefined";

    0 references
    public string Name
    {
        get
        {
            return name; // возвращаем значение свойства
        }
        set
        {
            name = value; // устанавливаем новое значение свойства
        }
    }
}
```

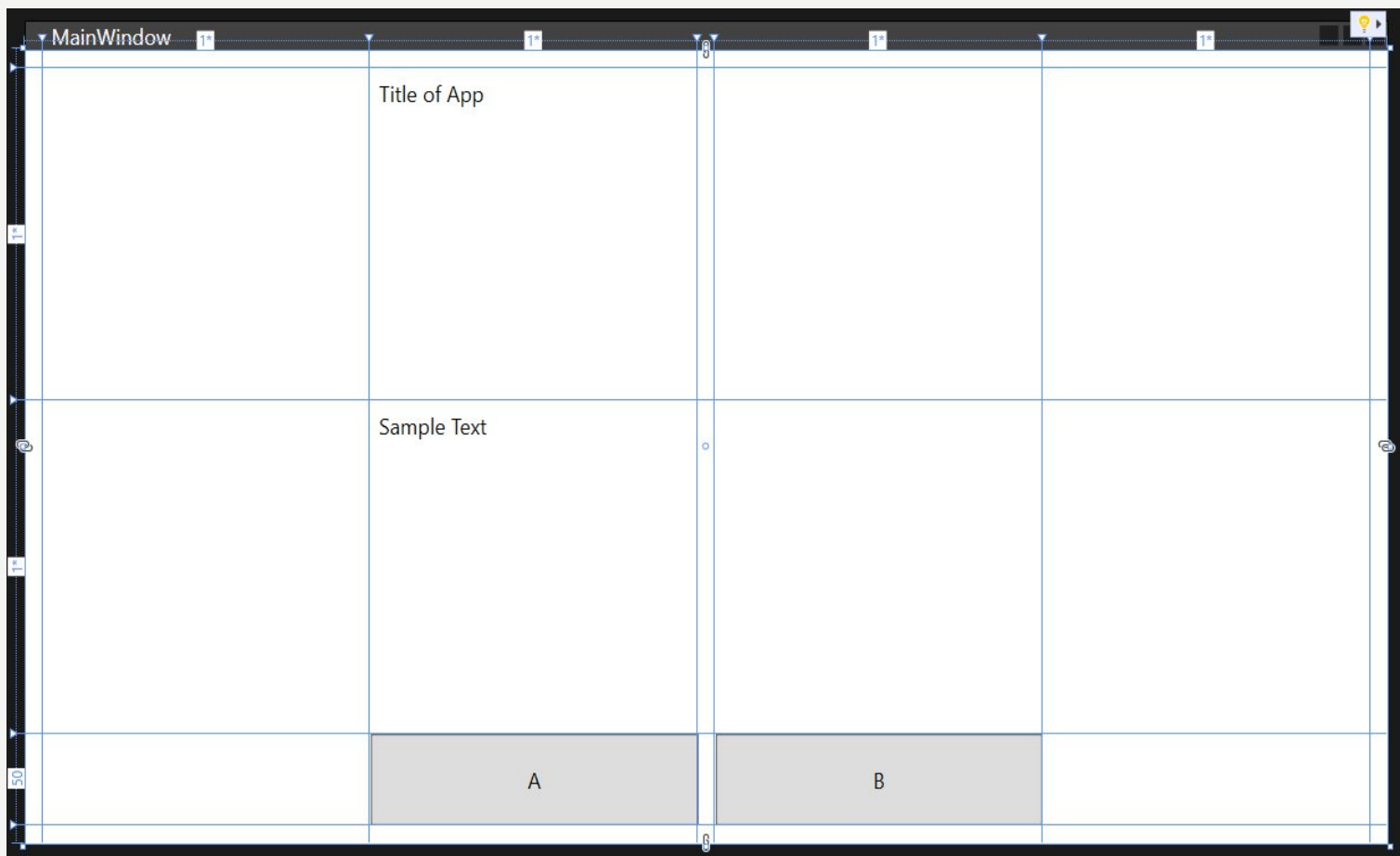
```
public class Cat : ISoundMaker
{
    2 references
    public void MakeSound()
    {
        Console.WriteLine("Мяу!");
    }
}
```



Связь с будущими курсами



Связь с будущими курсами



Login

Enter Password

Login

Currency Converter

Converted Currency

Enter Amount :

From :

To :

Convert

Clear

Игра с подбором пар животных

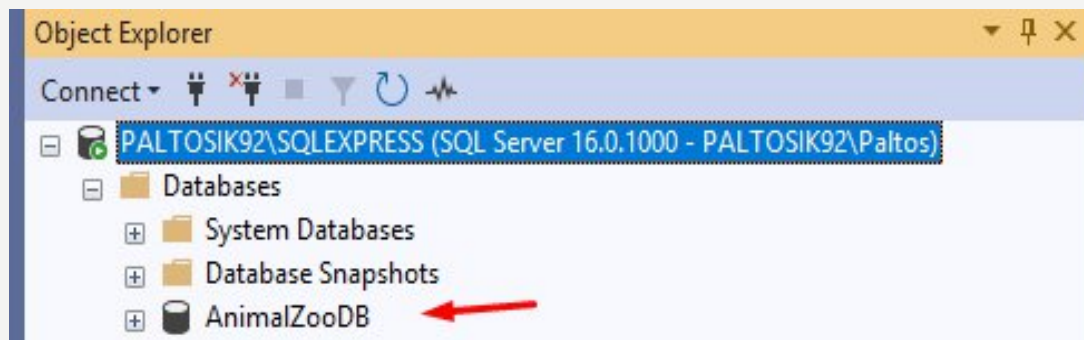
Game WPF

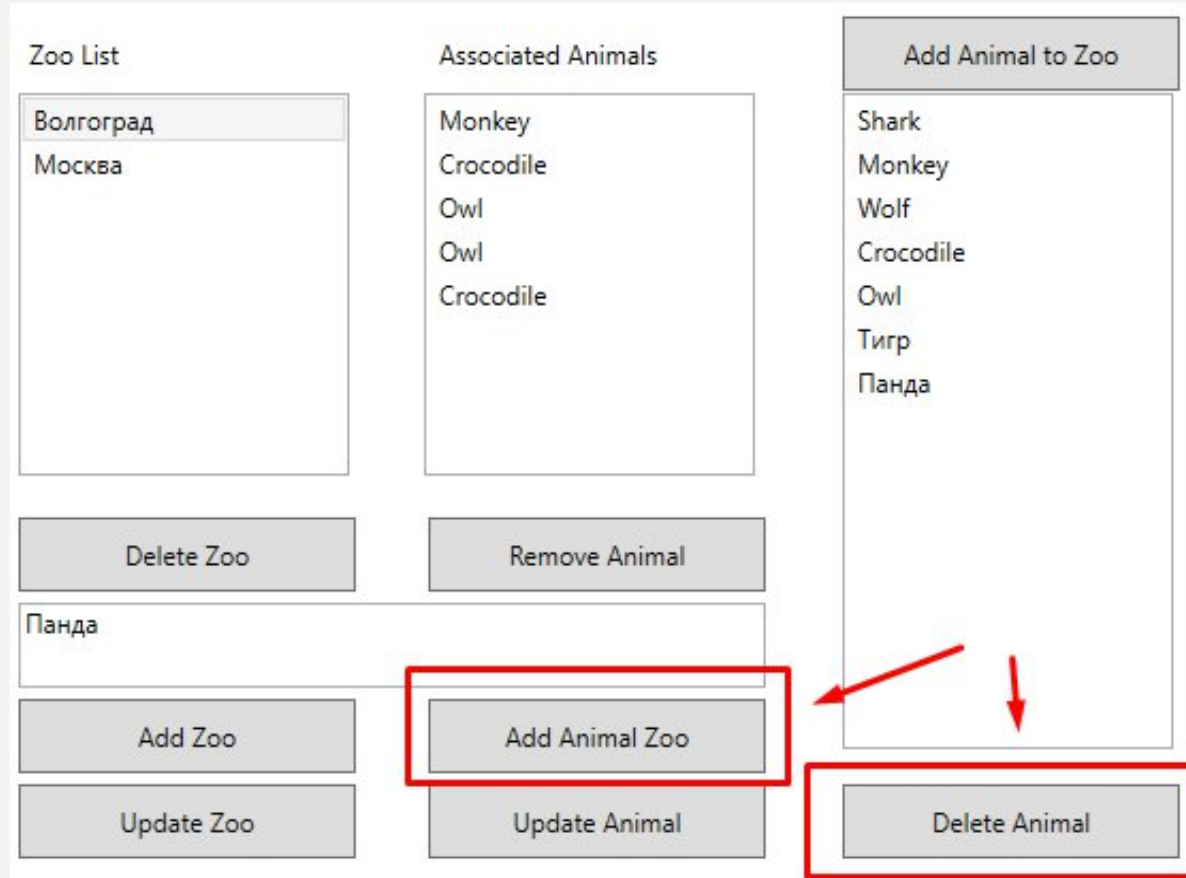
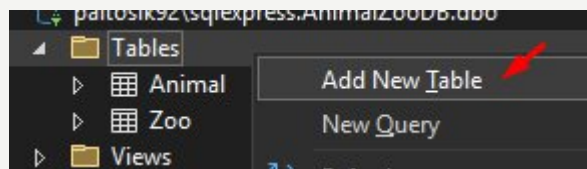
1,5s

На экране выводятся восемь пар животных, которые случайно распределяются в окне. Игрок должен щёлкать на двух одинаковых животных, чтобы пара исчезала из окна.

Таймер показывает сколько времени потребовалось для завершения игры.



```
CREATE TABLE [dbo].[Animal]
(
    [Id] INT NOT NULL PRIMARY KEY IDENTITY,
    [Name ] NVARCHAR(50) NOT NULL
)
```



dbo.Animal [Data] | X | dbo.Animal [Design]

Max Rows: 1000

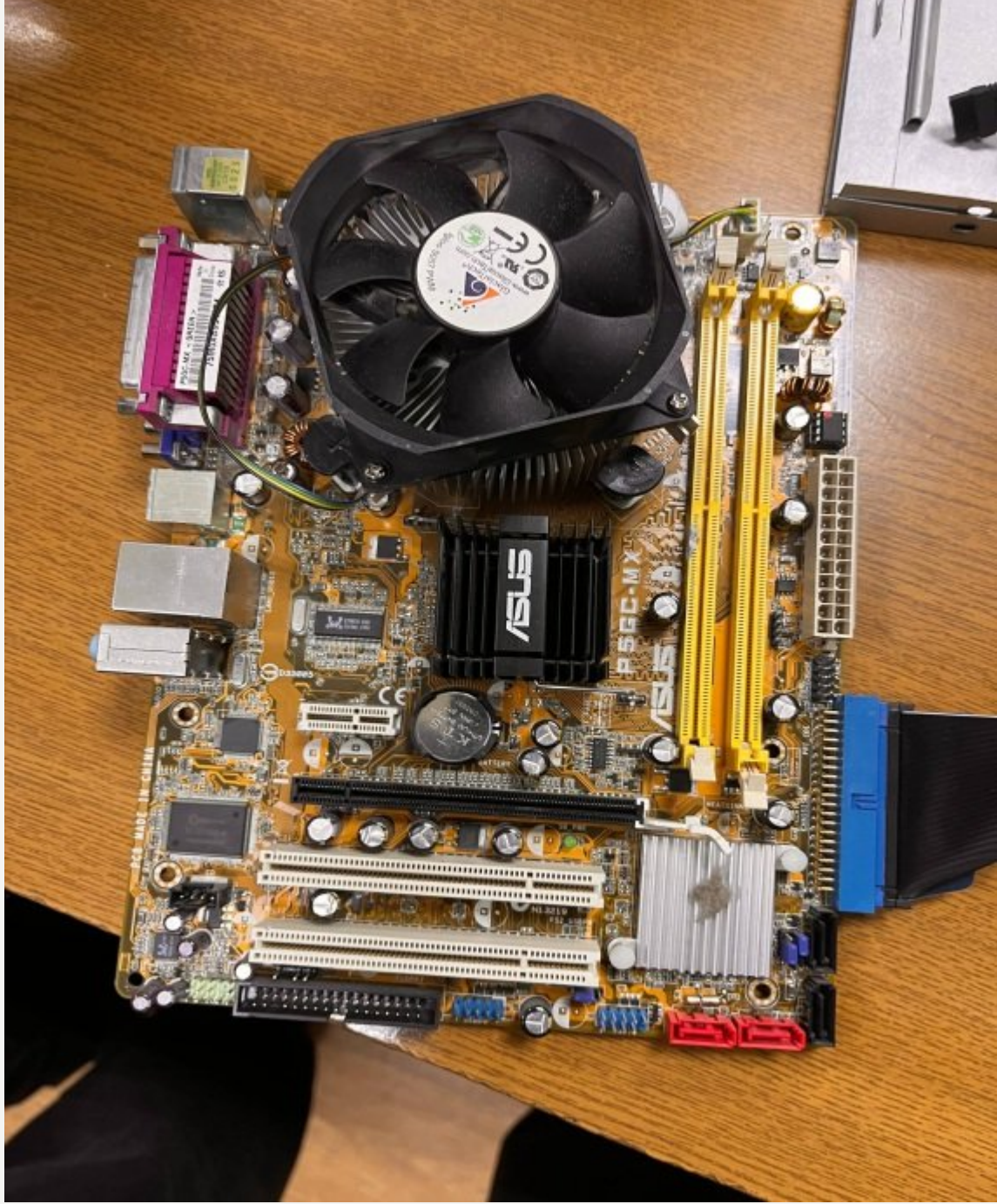
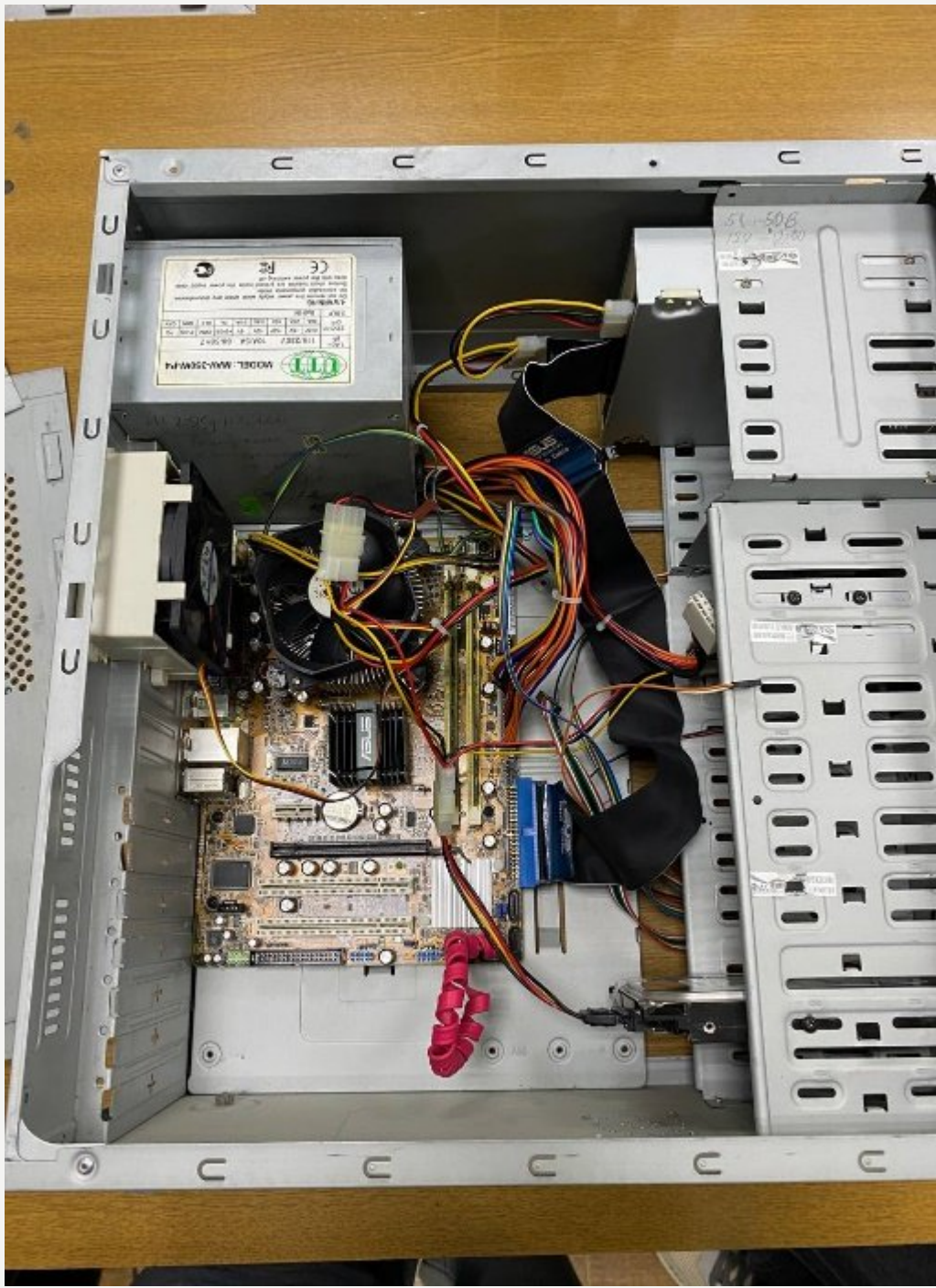
	Id	Name
	1	Shark
	2	Monkey
	3	Wolf
	4	Crocodile
	5	Owl
	6	Parrot
	NULL	NULL

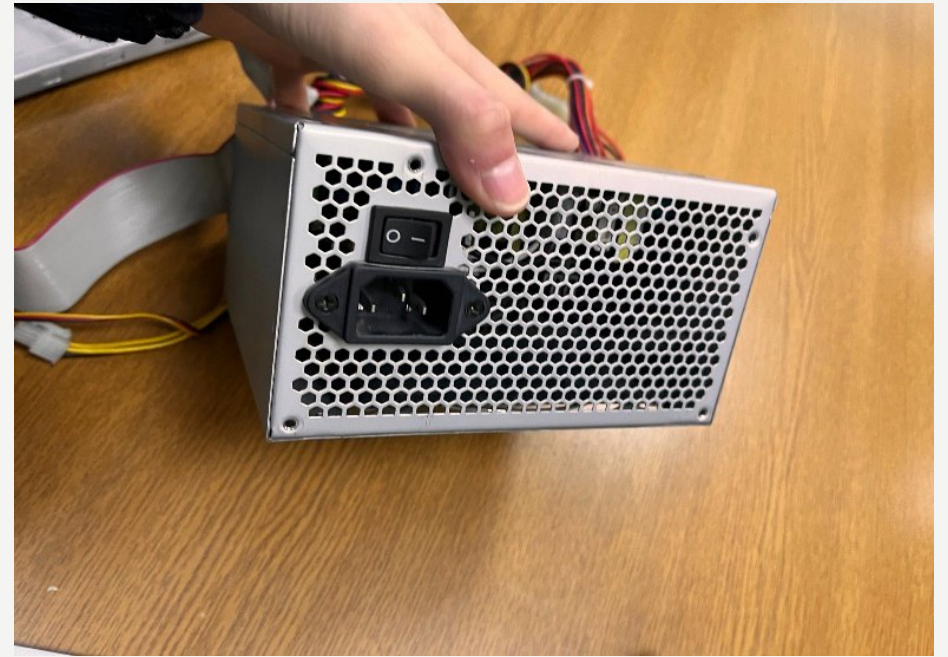
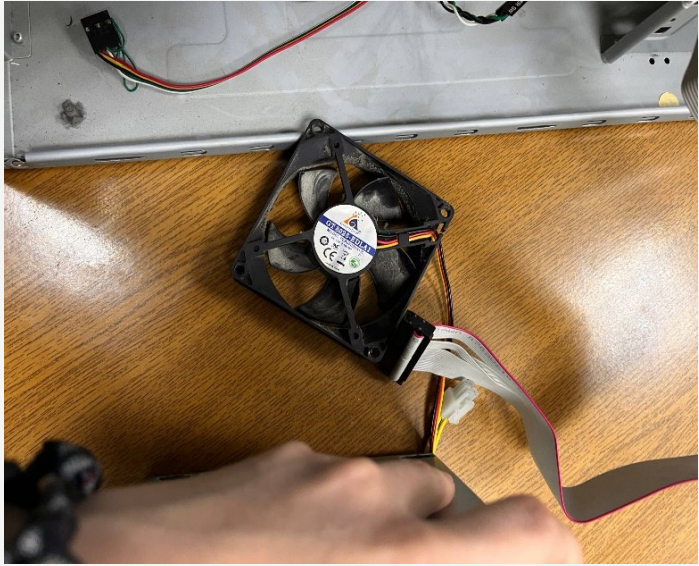
Живое железо

Что внутри корпуса

Что внутри системного блока



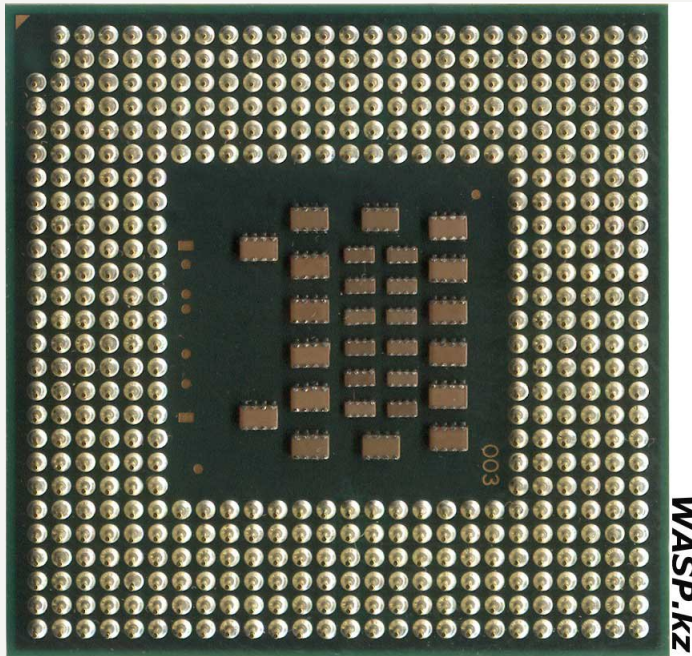




ПРОЦЕССОР (CPU) — сердце компьютера

Что вы видите:

- Сотни золотых контактов снизу
- Металлическая крышка сверху (теплораспределитель)
- Размер — примерно 4×4 сантиметра

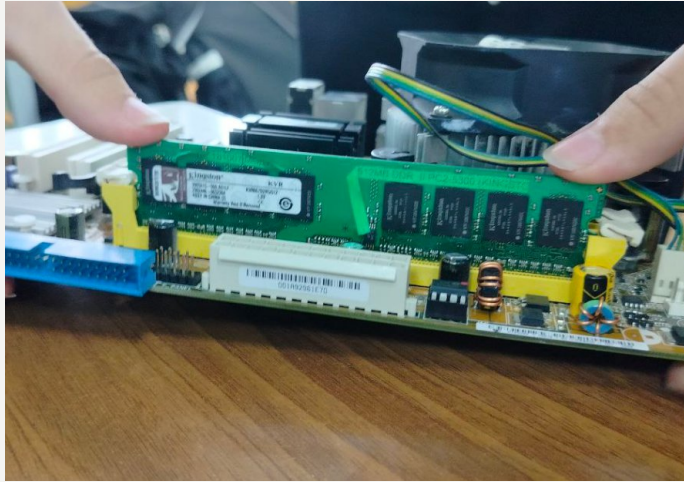


Что внутри (невидимо):

- Миллиарды транзисторов
- Ядро (отдельный "мозг")
- Кэш-память нескольких уровне



ОПЕРАТИВНАЯ ПАМЯТЬ И НАКОПИТЕЛЬ



RAM (оперативная память)

- 8–32 ГБ в типичном ПК
- Данные хранятся пока включён ПК
- Ваш код живёт здесь здесь во время работы



HDD (накопитель)

- 256–1000 ГБ
- Данные хранятся постоянно
- Ваши файлы живут здесь на диске

Инструменты

Какие инструменты будем использовать

ЧЕМ БУДЕМ РАБОТАТЬ

Git Bash - Терминал. Основной инструмент. Все команды — здесь.

```
Leontev@DESKTOP-6MFOUBL MINGW64 ~  
$ git --version  
git version 2.45.1.windows.1
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 ~  
$ git config --global user.name "Denis"
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 ~  
$ git config --global user.email denis.leontev92@yandex.ru
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 /c  
$ mkdir projectGit
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 /c/projectGit  
$ git init  
Initialized empty Git repository in C:/projectGit/.git/
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 /c/projectGit (master)  
$ git status  
On branch master  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   index.html  
    new file:   style/style.css
```

```
Leontev@DESKTOP-6MFOUBL MINGW64 /c/projectGit (master)  
$ git rm --cached index.html  
rm 'index.html'  
  
Leontev@DESKTOP-6MFOUBL MINGW64 /c/projectGit (master)  
$ git status  
On branch master  
  
No commits yet  
  
Changes to be committed:  
  (use "git rm --cached <file>..." to unstage)  
    new file:   style/style.css  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    index.html
```

GitHub - Хранилище вашего кода. Каждая лабораторная — в репозиторий.

Paltosik92 / flutter-offline-web-starter

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

flutter-offline-web-starterPublic

Pin

Watch0

Fork0

Star0

main1 Branch0 Tags

Go to file

Add file

<> Code

Paltosik92Initial commit0a97ab4 · 3 weeks ago🕒1 Commit

README.mdInitial commit3 weeks ago

bashrc_functionsInitial commit3 weeks ago

README

Flutter Offline Web Development Kit (Git Bash)

Набор Bash-функций для разработки Flutter Web-приложений без интернета. Предназначено для студентов и учебных проектов.

Что это даёт?

Возможность	Описание
🌐 Офлайн-сборка	Сборка Web-приложения без подключения к интернету
🔥 Локальный сервер	Быстрый запуск <code>python -m http.server</code> для тестирования
🧹 Автоматическая очистка	Умная очистка проекта с остановкой серверов
🚀 Быстрая перезагрузка	Одна команда: <code>clean → build → serve</code>

About

Набор Bash-функций для разработки Flutter Web-приложений без интернета.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors1

Paltosik92

Denis Leontev

Denis LeontevPaltosik92

Edit profile

6 followers · 0 following

Popular repositories

it-labsPublicHTML

lab10-ProcessPublicC#

flutter-offline-web-starterPublicНабор Bash-функций для разработки Flutter Web-приложений без интернета.

Lab07_CollectionsPublicC#

853 contributions in the last year

Contribution settings

2026

2025

2024

Contribution activity

March 2026

Created 173 commits in 48 repositories

Created 48 repositories

Show more activity

Seeing something unexpected? Take a look at the [GitHub profile guide](#).

Commit 226761d

 **Paltosik92** committed 5 days ago

Step 6: Dictionary – student grade journal

 **main**

1 parent [30b3dac](#) commit 226761d 

Filter files...

GradeJournal

Program.cs

Lab07_Collections.sln

 **2 files changed** +188 -109 lines changed

Search within code

GradeJournal/Program.cs  +164 -109  ...

...	@@ -1,121 +1,176 @@
1	- // // Шаг 4: Одномерные массивы
2	-
3	- // // Массив оценок группы
4	- // int[] grades = [85, 92, 78, 95, 88, 72, 90, 63, 77, 84];
5	- // string[] names = ["Алиса", "Борис", "Вера", "Глеб", "Диана",
6	- // "Егор", "Жанна", "Захар", "Ирина", "Кирилл"];
7	- // Console.WriteLine("Журнал оценок");
8	-
9	- // // Перебор с индексом
10	- // for (int i = 0; i < grades.Length; i++) {
11	- // Console.WriteLine(\$" {names[i],-10} - {grades[i]}");
1	+ // // // Шаг 4: Одномерные массивы
2	+
3	+ // // // Массив оценок группы
4	+ // // int[] grades = [85, 92, 78, 95, 88, 72, 90, 63, 77, 84];
5	+ // // string[] names = ["Алиса", "Борис", "Вера", "Глеб", "Диана",
6	+ // // "Егор", "Жанна", "Захар", "Ирина", "Кирилл"];

.NET SDK 10.0 - Платформа для запуска C#.

dotnet new / build / run

```
└─>$ dotnet --version  
10.0.104
```

Скачать .NET 10.0

<https://dotnet.microsoft.com/ru-ru/download/dotnet/10.0>

HxD - HEX-редактор. Смотрим внутрь файлов и программ.

Program.cs

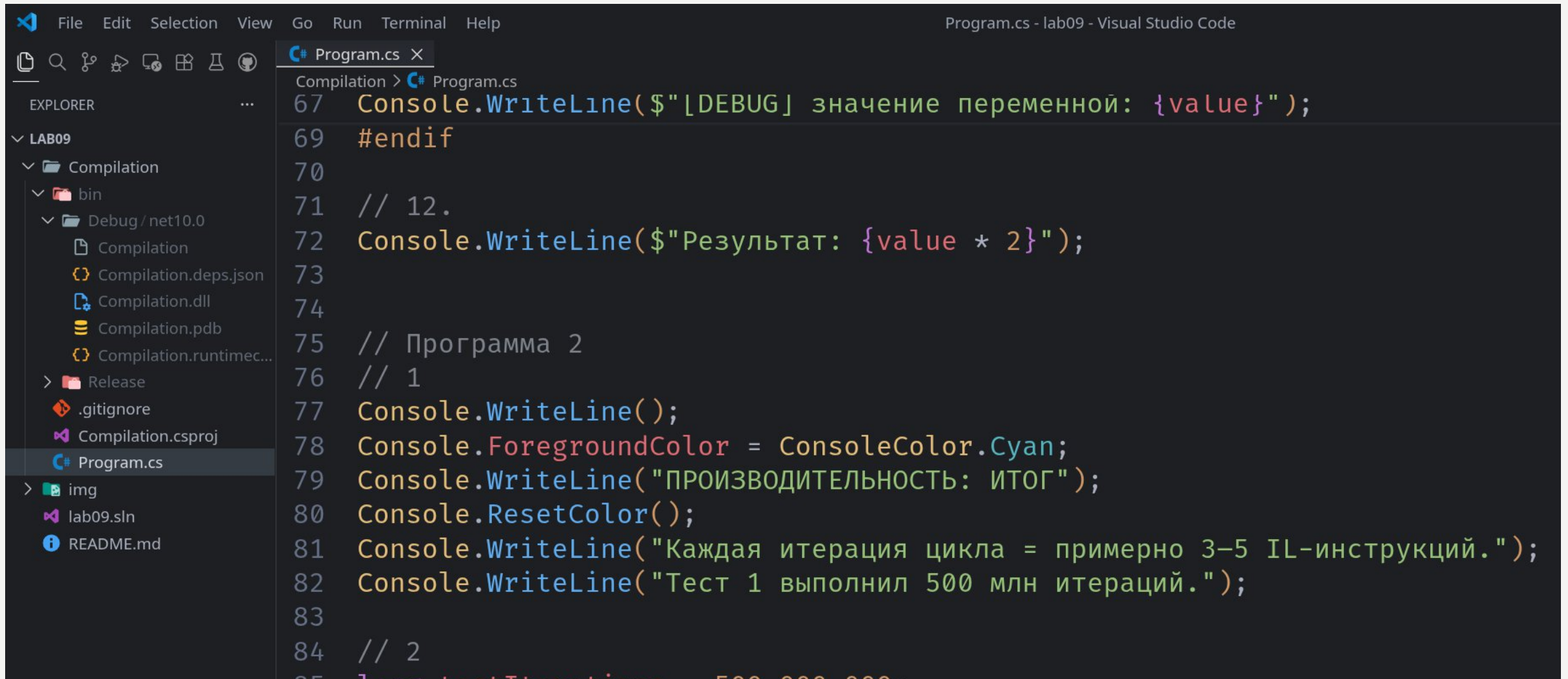
Compilation.dll
X

Compilation
>
bin
>
Debug
>
net10.0
>

Compilation.dll

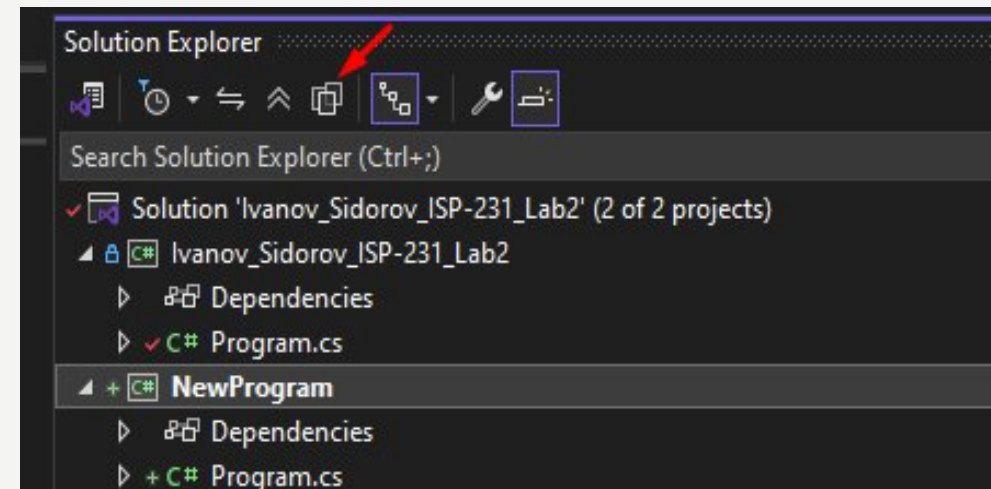
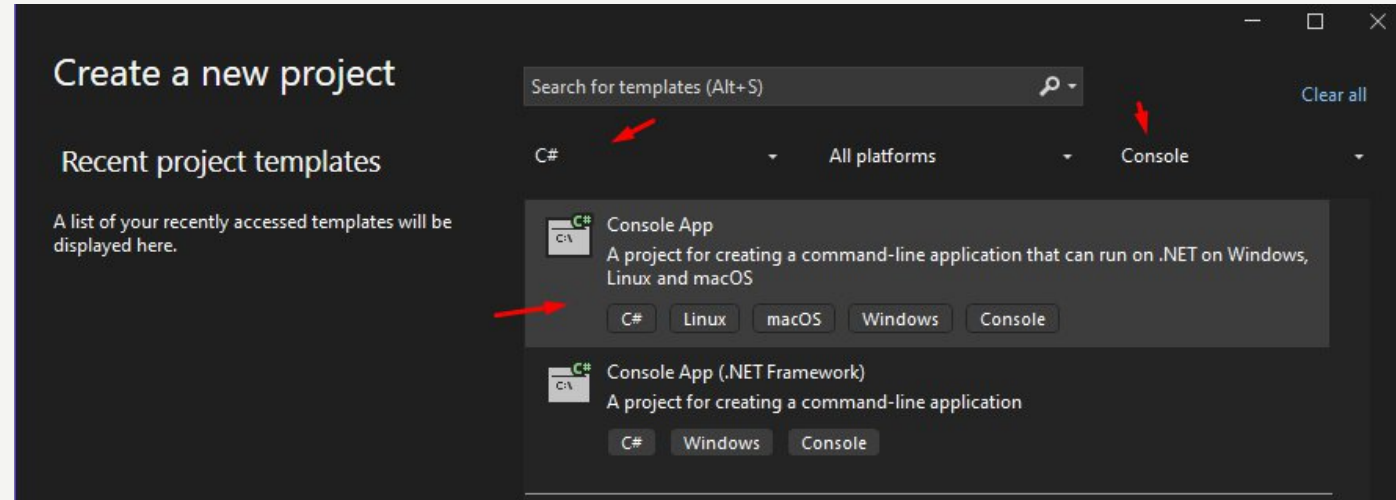
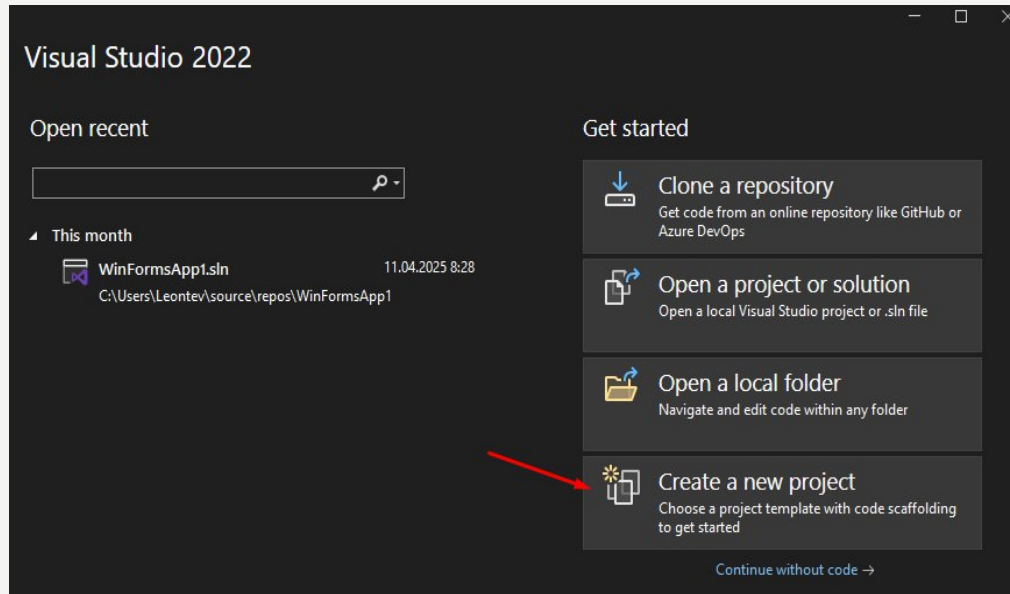
	0001020304050607	Decoded Text	Data Inspector	
00000000	4D5A900003000000	M Z	binary	01001101
00000008	04000000FFFF0000		octal	115
00000010	B800000000000000		uint8	77
00000018	4000000000000000	@	int8	77
00000020	0000000000000000		uint16	23117
00000028	0000000000000000		int16	23117
00000030	0000000000000000		uint24	9460301
00000038	0000000080000000		int24	-7316915
00000040	0E1FBA0E00B409CD		uint32	9460301
00000048	21B8014CCD215468	! . . L . ! T h	int32	9460301
00000050	69732070726F6772	i s p r o g r	uint64	12894362189
00000058	616D2063616E6E6F	a m c a n n o	int64	12894362189
00000060	742062652072756E	t b e r u n	ULEB128	77
00000068	20696E20444F5320	i n D O S	SLEB128	-51
00000070	6D6F64652E0D0D0A	m o d e	float16	201.625
00000078	2400000000000000	\$	bfloat16	14425592556421120
00000080	504500004C010300	P E . . L . . .	float32	1.325670526335053e-38
00000088	0724F1B300000000	. \$	float64	6.3706613826e-314

VS Code - Редактор кода. Лёгкий, быстрый, бесплатный.



Почему не **Visual Studio** или **Rider**?

- На слабых ПК грузится 5–10 минут.
- VS Code запускается за 2 секунды.



PYTHON VS C#

Мост между языками

Производительность · Компиляция · Типизация

Одна программа на двух языках

ВЫ УЖЕ УМЕЕТЕ ПРОГРАММИРОВАТЬ

Python	C#
<pre>name = input("Имя: ")</pre>	<pre>Console.Write("Имя: "); string name = Console.ReadLine();</pre>
<pre>age = int(input("Возраст: "))</pre>	<pre>Console.Write("Возраст: "); int age = int.Parse(Console.ReadLine());</pre>
<pre>print(f"Через 10 лет тебе будет {age+10}")</pre>	<pre>Console.WriteLine(\$"Через 10 лет тебе будет {age + 10}");</pre>

Логика одинаковая. Только записано по-другому.

Три отличия которые важны

ОТЛИЧИЕ 1: Явные типы данных

```
Python:   name = "Алёша"      ← Python догадывается сам  
C#:       string name = "Алёша" ← мы говорим явно
```

- Зачем? Компилятор знает сколько памяти выделить заранее. Программа быстрее и предсказуемее.
- Ошибки типов находятся ДО запуска, не во время.

ОТЛИЧИЕ 2: Компиляция перед запуском

```
Python:   интерпретатор читает строку за строкой  
C#:       компилятор проверяет весь код, потом запускает
```

Зачем? Ошибки видны сразу. И работает быстрее в 10–50 раз.

ОТЛИЧИЕ 3: Фигурные скобки вместо отступов

Python: отступы задают структуру

C#: фигурные скобки { } задают структуру

Зачем? Явно. Нет путаницы с табами и пробелами.

Производительность

Во сколько раз быстрее?

ПРОИЗВОДИТЕЛЬНОСТЬ

Задача: сложить числа от 1 до 100 миллионов

Python

~4–5 сек

интерпретирует построчно

C#

~80–120 мс

скомпилирован в машинный код

Разница: в 40–50 раз

Почему такая разница?

- **Python** — интерпретирует каждую строку во время работы программы
- **C#** — скомпилирован заранее в машинный код; процессор выполняет его напрямую

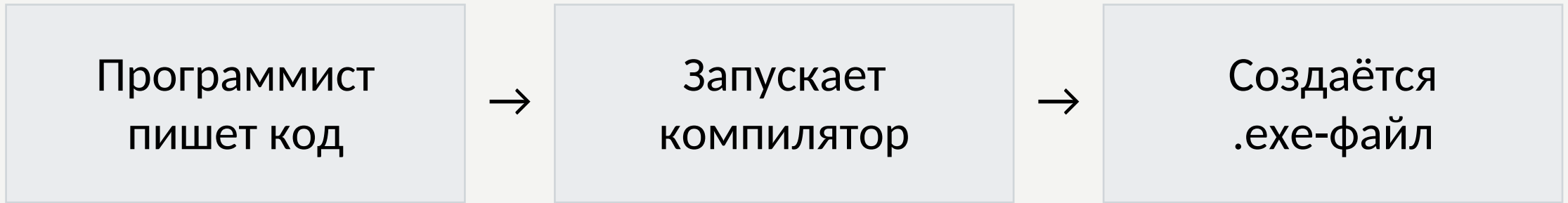
Убедитесь сами на практической работе (циклы и производительность)

Типы языков

Компилируемые и интерпретируемые

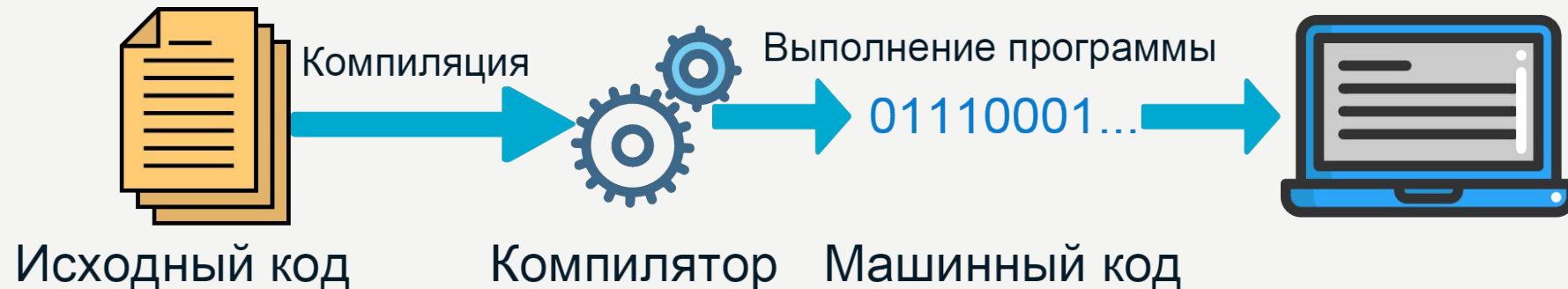
Компилируемые языки

Компилятор переводит весь код в машинный сразу — до первого запуска программы.



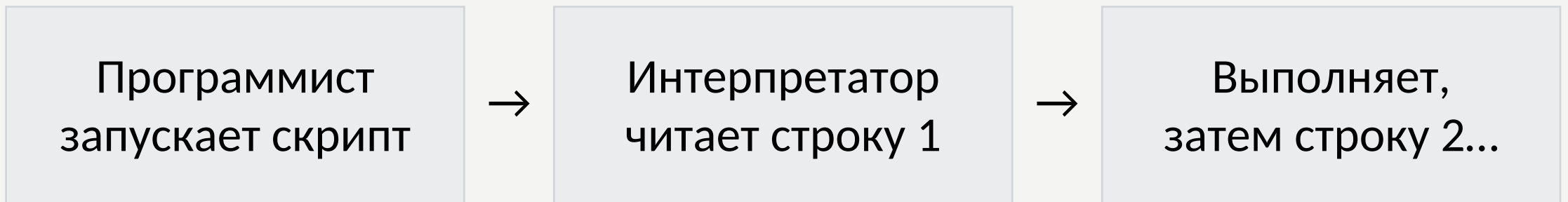
Примеры: C, C++, Java, Swift, Go, C# (не совсем)

Программа компилируется один раз — каждый следующий запуск выполняется быстро.



Интерпретируемые языки

Интерпретатор переводит код построчно — каждый раз при запуске программы.



Примеры: Python, PHP, JavaScript

Нет этапа компиляции — но каждый запуск медленнее из-за построчного перевода.

Компилируемые vs Интерпретируемые

Компилируемые

- Высокая скорость выполнения
- Ошибки типов выявляются до запуска
- Нужен этап компиляции
- Дольше на старте разработки

Интерпретируемые

- Быстрый старт разработки
- Не требует компиляции
- Медленнее в выполнении
- Ошибки могут появиться только при запуске

Сравнение программы “Hello world” на двух языках

```
print("Hello world!")
```

На Python

```
using System;

namespace HelloWorld
{
    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Hello world!");
        }
    }
}
```

На C#

С# vs Python: ключевые различия

Характеристика	С#	Python
Скорость выполнения	Выше (JIT-компиляция)	Ниже (интерпретация)
Поддержка ООП	Очень сильная, строгая	Гибкая, менее строгая
Запуск программы	Компиляция перед выполнением	Выполнение сразу
Применение	Корпоративные приложения, игры (Unity), backend	Скрипты, анализ данных, ML, веб

Типизация

Статическая и динамическая

Что такое типизация?

Набор правил, по которым язык программирования классифицирует данные.

Без строгой типизации переменная может сначала хранить число, а потом строку. Это ускоряет написание кода — но в таком коде проще запутаться.

Именно поэтому языки выбирают один из двух подходов: статический или динамический.

Статическая типизация

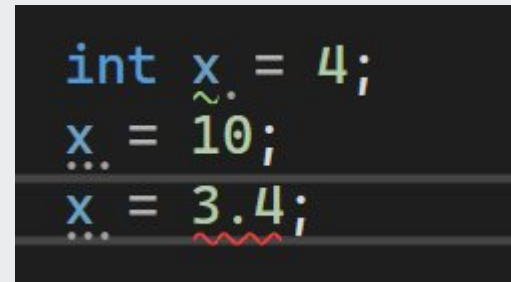
Типы данных определяются до запуска программы — во время компиляции.

Особенности

- Тип переменной задаётся явно
- Ошибки типов — до запуска
- Надёжность и читаемость кода выше

Примеры языков

- Java
- C++
- Swift
- C#



```
int x = 4;  
x = 10;  
x = 3.4;
```

Динамическая типизация

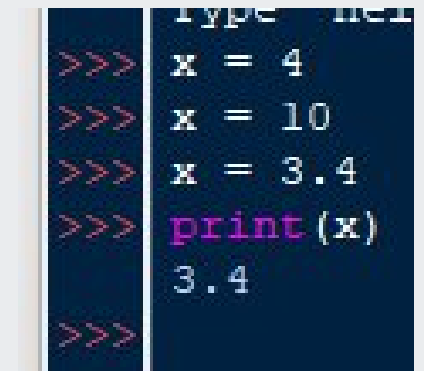
Тип переменной определяется во время выполнения. Объявлять тип явно не нужно — интерпретатор сам это делает.

Особенности

- Интерпретатор определяет тип сам
- Код пишется быстрее
- Ошибки типов — только при запуске

Примеры языков

- Python
- JavaScript
- Ruby



```
>>> x = 4
>>> x = 10
>>> x = 3.4
>>> print(x)
3.4
>>>
```


Но бывает и исключения (динамический тип данных в C#):

```
static void Main(string[] args)
{
    dynamic x = 4;
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
    x = "hello";
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
    x = 3.4;
    Console.WriteLine(x);
    Console.WriteLine(x.GetType());
}
```

Консоль отладки Microsoft V

```
4
System.Int32
hello
System.String
3,4
System.Double
C:\Users\Palto
```

Основные конструкции

Python vs C#

Условия (проверка числа)

```
num = 12
if num < 10:
    print("Меньше 10")
elif 10 <= num <= 15:
    print("От 10 до 15")
else:
    print("Больше 15")
```

На Python

```
int num = 12;
if (num < 10)
{
    Console.WriteLine("Меньше 10");
}
else if (num >= 10 && num <= 15)
{
    Console.WriteLine("От 10 до 15");
}
else
{
    Console.WriteLine("Больше 15");
}
```

На C#

Цикл for (вывод чисел от 1 до 10)

```
for i in range(1, 11):  
    print(i)
```

На Python

```
for (int i = 1; i <= 10; i++)  
{  
    Console.WriteLine(i);  
}
```

На C#

Функция/Метод для суммы двух чисел

```
def sum_numbers(a, b):  
    return a + b  
  
result = sum_numbers(5, 3)  
print(result) # 8
```

На Python

```
static int SumNumbers(int a, int b)  
{  
    return a + b;  
}  
  
int result = SumNumbers(5, 3);  
Console.WriteLine(result); // 8
```

На C#

Простой класс Студент

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Привет, меня зовут {self.name}!")

# Использование
student = Student("Алексей", 20)
student.greet() # Привет, меня зовут Алексей!
```

На Python

На C#

```
internal class Program
{
    0 references
    static void Main(string[] args)
    {
        // Использование
        Student student = new("Алексей", 20);
        student.Greet(); // Привет, меня зовут Алексей!
    }
}

3 references
public class Student
{
    2 references
    public string Name { get; set; }
    1 reference
    public int Age { get; set; }

    1 reference
    public Student(string name, int age)
    {
        Name = name;
        Age = age;
    }

    1 reference
    public void Greet()
    {
        Console.WriteLine($"Привет, меня зовут {Name}!");
    }
}
```


ЧАСТЬ 2

Живая демонстрация

Видео-демо №1: Среда разработки

ДЕМОНСТРАЦИЯ №1

Настройка среды разработки

Что покажу:

- Как выглядит Git Bash
- Основные команды терминала
- Создание проекта через dotnet CLI
- VS Code изнутри
- Первая программа

На **ЛР №1** вы сделаете это сами — шаг за шагом.

Видео

Видео-демо №2: Git и GitHub

Зачем GitHub

GITHUB — ВАШ ПУБЛИЧНЫЙ ПОРТФОЛИО

Что это: Облачное хранилище для кода с историей изменений

Зачем вам:

- ✓ Сдача лабораторных — каждая работа в свой репозиторий
- ✓ История изменений — можно откатиться к любой версии
- ✓ Портфолио — работодатели смотрят GitHub при найме
- ✓ Командная работа — итоговый проект делаете вместе

Реальность:

- ✓ На каждом собеседовании: "Покажите ваш GitHub"
- ✓ Пустой GitHub — минус к впечатлению
- ✓ GitHub с 15 лабораторными — уже есть что показать



Переход к демо №2

Git и GitHub — первый коммит

Что покажу:

- **git init** — инициализация репозитория
- **git add / commit** — сохранить изменения
- **Создание репозитория** на GitHub
- **git push** — отправить на GitHub

Четыре команды которые будете использовать каждую лабораторную.

Видео

Итог демо №2

РАБОЧИЙ ЦИКЛ КАЖДОЙ ЛАБОРАТОРНОЙ

1. Создать папку проекта
2. `dotnet new console`
3. `git init`
4. Написать код
5. `dotnet run` ← проверить что работает
6. `git add .`
7. `git commit -m "описание что сделал"`
8. `git push`

Это войдёт в привычку после первых двух лабораторных.

Шпаргалка команд

ТЕРМИНАЛ (Git Bash)

<code>pwd</code>	— где я сейчас?
<code>ls</code>	— что в этой папке?
<code>cd папка</code>	— перейти в папку
<code>cd ..</code>	— выйти на уровень выше
<code>mkdir имя</code>	— создать папку

DOTNET

<code>dotnet new console</code>	— создать C# проект
<code>dotnet run</code>	— скомпилировать и запустить
<code>dotnet build</code>	— только скомпилировать

GIT

<code>git init</code>	— начать отслеживать папку
<code>git status</code>	— что изменилось?
<code>git add .</code>	— подготовить все файлы
<code>git commit -m "..."</code>	— сохранить снимок
<code>git push</code>	— отправить на GitHub

**Спасибо
за внимание**

Лекция №1 · Вводная лекция · инструменты, железо и первый код